
```
class PointWithSlots():
    # Define slots for only two instance variables
    __slots__ = ['x', 'y']

    def __init__(self, x, y):
        self.x = x
        self.y = y
        print(x, y)
```

These two classes will work identically, but objects instantiated from `PointWithSlots` will take up considerably less memory. To demonstrate the difference, we'll add this line to the end of the `__init__()` method of both classes:

```
    # Try to create an additional instance variable
    self.color = 'black'
```

Now when we try to instantiate an object from both classes, the `Point` class has no problem adding another instance variable, but the `PointWithSlots` class fails with the following error:

```
AttributeError: 'PointWithSlots' object has no attribute 'color'
```

Using slots is highly memory-efficient at the expense of a loss of dynamic instance variables. If you're dealing with a very large number of objects from a class, this trade-off may very well be worthwhile.

Summary

This chapter focused on a few concepts that didn't readily fit in the previous chapters. First, I discussed the circumstances under which you might want to delete an object. We looked at reference counts and how they track how many variables refer to the same object, which led to a discussion of object lifetimes and garbage collection. When the reference count goes to zero, the object is available for garbage collection. If a class has a `__del__()` method, then any objects created from the class can use the `__del__()` method for any cleanup that they might want to do.